

# 关于一道有趣的题目设计

## 参考

<https://github.com/ading2210/linuxpdf>

以及25年9月TPCTF的linuxpdf题目

## 先介绍原理

- PDF中的JavaScript：PDF格式支持JavaScript。虽然浏览器中的PDF引擎只支持部分API，但这足以运行代码
- C到JavaScript的编译：Linux是用C语言写的。项目利用一个叫 Emscripten 的旧版本工具，将C代码编译成 asm.js。asm.js是JavaScript的一个子集，能在PDF中高效运行
- 运行RISC-V模拟器：核心是 TinyEMU，一个轻量级的RISC-V系统模拟器。开发者把它编译成 asm.js后嵌入PDF，模拟器启动后便会加载并运行一个极简的Linux内核

```
git clone https://github.com/ading2210/linuxpdf
cd linuxpdf
pip install -r requirements.txt
```

```
sudo apt update
```

```
sudo apt install -y gcc-riscv64-linux-gnu qemu-user qemu-user-static binfmt-support
```

随即编写好所需的 challenge.c

```
mkdir -p challenge
cd challenge
riscv64-linux-gnu-gcc -static -O2 -o chal64 chal.c
file chal64          # 确认是 RISC-V 64-bit
qemu-riscv64 ./chal64 # 本地测试
cd ..
```

由于我的wsl2是x86\_64，所以我更改了build.sh里面的 BITS

但是

```
./build.sh
```

后发生问题

但是还有地方也是架构不匹配发生问题

Alpine 根文件系统的所有程序都是针对 RISC-V 编译的

```
# 复制 QEMU 静态模拟器
sudo cp /usr/bin/qemu-riscv64-static build/alpine/usr/bin/

# 设置 DNS
echo -e "nameserver 1.1.1.1\nnameserver 8.8.8.8" | sudo tee build/alpine/etc/resolv.conf

# 设置软件源
sudo tee build/alpine/etc/apk/repositories > /dev/null << EOF
https://dl-cdn.alpinelinux.org/alpine/v3.24/main
https://dl-cdn.alpinelinux.org/alpine/v3.24/community
EOF

# 安装必要软件
sudo chroot build/alpine /usr/bin/qemu-riscv64-static /sbin/apk update
sudo chroot build/alpine /usr/bin/qemu-riscv64-static /sbin/apk add --no-cache agetty nano httpd

# 配置自动登录和启动脚本
echo "tty1::respawn:/sbin/agetty --autologin root tty0 linux" | sudo tee build/alpine/etc/inittab

sudo tee build/alpine/root/.profile > /dev/null << 'EOF'
#!/bin/sh
mount -t proc none /proc
mount -t sysfs none /sys
hostname -F /etc/hostname
echo 'VM boot complete' > /dev/hvc0
EOF
sudo chmod +x build/alpine/root/.profile

echo "linuxpdf" | sudo tee build/alpine/etc/hostname
echo -n "" | sudo tee build/alpine/etc/motd
```

以上将 QEMU 静态模拟器放入 chroot 环境，确保 chroot 内也可以执行 RISC-V 程序，并配置 DNS，设置 Alpine 软件源

```
sudo cp challenge/chal64 build/alpine/root/chal
sudo chmod +x build/alpine/root/chal
```

将文件放入后重新打包

```
rm -rf build/files
./build.sh
```

## 题解

```
data = file_path.read_bytes()
data_compressed = zlib.compress(data, level=9)
data_b64 = base64.b64encode(data_compressed)
files[rel_path] = data_b64.decode()

js = js_path.read_text()
js = js.replace("__files_data__", json.dumps(files))
```

阅读源代码知道embed\_files.py将files经过zlib+base64后生成一个字典并替换了\_\_files\_data\_\_

```
var embedded_files = __files_data__;
for (let filename in embedded_files) {
    embedded_files[filename] = pako.inflate(b64_to_uint8array(embedded_files[filename]));
}
```

在file\_template.js中又可以看见\_\_files\_data\_\_被传入这里

```
4
5 build_files
6 cp vm_${BITS}.cfg build/vm/bbl${BITS}.bin build/vm/kernel-riscv${BITS}.bin build/files
7
8 python3 embed_files.py file_template.js build/files/ build/files.js
9 cat build/pako.min.js build/files.js pdflinux.js tinyemu/js/riscvemum${BITS}.js > out/compiled.js
10
11 python3 gen_pdf.py out/compiled.js out/linux.pdf
12 cp web/* out
```

在这里又可以看见将四个 JS 模块按顺序拼接成一个完整的文件，再将拼接好的 JavaScript 代码注入到一个 PDF 文件中，生成最终的 linux.pdf

因为这个字典是js啊，我们以文本模式打开的时候可以读到，直接定位就行

```

import base64
import zlib
import os
import re

def base64_decode_and_inflate(encoded_data):
    try:
        compressed_data = base64.b64decode(encoded_data)

        # 尝试 zlib
        try:
            decompressed_data = zlib.decompress(compressed_data)
            return decompressed_data
        except zlib.error:
            pass

        # 尝试 gzip
        try:
            decompressed_data = zlib.decompress(compressed_data, 16 + zlib.MAX_WBITS)
            return decompressed_data
        except zlib.error:
            pass

        # 尝试 raw deflate
        try:
            decompressed_data = zlib.decompress(compressed_data, -zlib.MAX_WBITS)
            return decompressed_data
        except zlib.error as e:
            print(f"解压缩失败: {e}")
            return None

    except Exception as e:
        print(f"解码失败: {e}")
        return None

def main():
    pdf_path = r"path" # 改成你的 PDF 路径
    output_dir = "extracted" # 提取到这个文件夹

    with open(pdf_path, "r", encoding="utf-8", errors="ignore") as f:
        content = f.read()

```

```

# 定位数据段（根据你原来的逻辑）
start_index = content.find('"vm_64.cfg"')
if start_index == -1:
    print("找不到 vm_64.cfg, 请检查 PDF 或搜索关键字")
    return

end_marker = "for \\(let filename i"
end_index = content.find(end_marker, start_index)
if end_index == -1:
    print("找不到结束标记")
    return

data_section = content[start_index:end_index - 4]

# 按 ", " 分割每条记录
items = data_section.split(", ")

success = 0
failed = 0

for item in items:
    if ": " not in item:
        continue

    try:
        path_part, data_part = item.split(": ", 1)
        file_path = path_part.replace("'", "").strip()
        b64_data = data_part.replace("'", "").strip()

        if not file_path or not b64_data:
            continue

        decoded = base64_decode_and_inflate(b64_data)
        if decoded is None:
            print(f"[失败] {file_path}")
            failed += 1
            continue

        # 完整输出路径
        full_path = os.path.join(output_dir, file_path)

        # 自动创建目录
        dir_name = os.path.dirname(full_path)

```

```
    if dir_name:
        os.makedirs(dir_name, exist_ok=True)

    with open(full_path, "wb") as out:
        out.write(decoded)

    print(f"[成功] {full_path} ({len(decoded)} bytes)")
    success += 1

except Exception as e:
    print(f"[异常] {item[:50]}... -> {e}")
    failed += 1

print("\n===== 完成 =====")
print(f"成功: {success}")
print(f"失败: {failed}")
print(f"文件保存在: {os.path.abspath(output_dir)}")

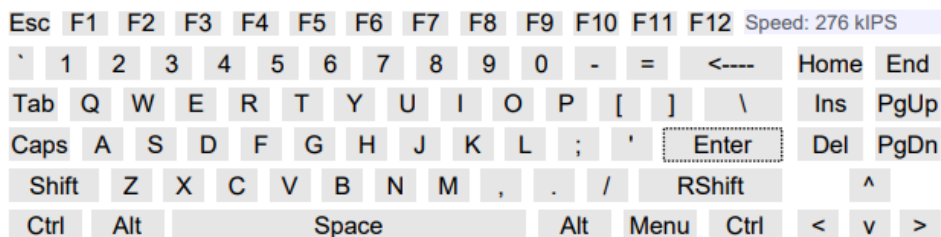
if __name__ == "__main__":
    main()
```

然后发现dump出来一堆东西怎么定位呢

```
localhost login: root (automatic login)
login[34]: root login on 'UNKNOWN'
linuxpdf:~# ./chal
Input: _
```

```
loading root/files/000000000000008d
loading root/files/0000000000000072
loading root/files/0000000000000068
loading root/files/0000000000000070
loading root/files/0000000000000051
loading root/files/0000000000000011
loading root/files/0000000000000013
loading root/files/0000000000000046
loading root/files/000000000000000a
loading root/files/0000000000000044
down: .
up: .
down: /
up: /
down: c
up: c
down: h
up: h
down: a
up: a
down: l
up: l
down: Enter
up: Enter
loading root/files/000000000000000b
```

## LinuxPDF



Pressed:

Type here for keyboard inputs.

Source code: <https://github.com/ading2210/linuxpdf>

Note: This PDF only works in Chromium-based browsers.

To send keystrokes to the VM, use the buttons or type in the box above.

细看左下角可以看到load了什么，直接对着找出来就行

然后就是解里面的题了